

AIにコードを直させる前に必要だったこと

Jira・Slack・MCPをつないだ自動修正基盤 Qualix の試行錯誤

manattan

今日話すこと

- なぜ Qualix を作ろうと思ったのか
- Jira の品管チケットを起点に、何を自動化したかったのか
- 現状のアーキテクチャ
- 実装よりも難しかった、権限・通知・承認・運用の話
- 今後のデザイン

背景: コードはAIが書けるようになった

Claude Code や GitHub Copilot によって、コード修正そのものはかなり速くなった。

ただし、現場のバグ対応では、コードを書く前にまだ人間がやっていることが多い。

- チケットに気づく
- 仕様を追う
- ログを見る
- 関連repoを探す
- 原因を推定する
- AIに適切に依頼する

問題意識

AIに依頼するまでがまだ重い

コード生成AIが爆速で改善してくれる一方で、そこに到達するまでの調査・整理・判断に時間がかかっていた。

特に品管チケットでは、以下が起きやすい。

- 重大なチケットに気づくのが遅れる
- どのチーム・repoが見るべきか曖昧
- Jiraだけでは修正に必要な情報が足りない
- Slack、Grafana、GitHub、仕様が分散している

目指した世界

Jiraチケットを起点に、AIが初動調査から修正案まで進める

Jira 品管チケット

- > Qualix
- > チケット内容を解析
- > 対象サービス / repo / 環境を推定
- > Jira / Grafana / GitHub / Slack から文脈を収集
- > 原因仮説と修正方針を作成
- > GitHub Issue / PR を作成
- > 人間がレビュー

Qualixとは

Jira のチケットを AI エージェントが自動で調査し、GitHub Issue や将来的な修正 PR へつなげるシステム。

現在の主な対象は、品管チケット。

- Jira Webhookで起動
- AI AgentがMCP経由で情報収集
- Grafana Lokiからログを取得
- GitHub Issueを作成
- Jiraに結果をコメント
- 将来的にはCopilot / PR作成まで拡張

現状アーキテクチャ

```
Jira Webhook
  |
  v
Echo Server (/webhooks/jira)
  |
  v
Ticket Filter / Classifier
  |
  v
OpenAI Agent Loop
  |
  +-- Jira MCP
  +-- Grafana MCP
  +-- GitHub MCP
  |
  v
GitHub Issue / Jira Comment
```

実装の現在地

コードとしては、PoCから実運用手前まで形になっている。

- Jira Webhook受信
- HMAC-SHA256署名検証
- チケットフィルタ
- 軽量AI分類
- MCP tool収集
- allowed / denied toolsによる制御
- OpenAI Agent loop
- service catalogによるrepo / namespace / datasource制御

一方で、運用に載せるには調整が必要。

重要な設計: AIに自由生成させすぎない

AIにrepo名やnamespaceを自由に作らせると危険。

そのため、Qualixでは YAML の service catalog を single source of truth として扱う。

```
repositories:  
  - service: qualix  
    repository: manattan/qualix  
    jira_components: ["qualix"]  
    environments:  
      qa:  
        namespace: qualix-qa  
        grafana_datasource: loki-qa
```

AIには「選ばせる」。決定論的に解決できる値はコードと設定で縛る。

難所1: どこを直すべきかを推定する

自動修正の前に、まずこの推定が必要。

- どのQAホストで起きているか
- qa / staging / prod のどれか
- サーバー起因か、クライアント起因か
- どのプロダクト領域か
- どのrepoを見るべきか
- どのログを見るべきか
- 仕様なのか、バグなのか

自動修正の難しさは、パッチを書くことよりも、どこを直すべきかに辿り着くことだった。

難所2: 情報はJiraだけじゃない

Jiraチケットは入口として優秀だが、修正に必要な情報は分散している。

- Jira: 現象、再現手順、優先度、コメント
- Slack: 補足説明、現場の会話、暫定対応
- Grafana / Loki: 実際のエラー、頻度、発生時刻
- GitHub: 実装、過去PR、関連Issue
- Docs: 仕様、運用ルール

AIにコードを書かせる前に、AIが判断できる文脈を揃える必要がある。

難所3: 権限と社内調整

コードはほぼできたが、運用に載せる段階で別の壁が見えてきた。

- Jiraには秘匿情報が多い
- Jira API / MCP利用には許可が必要
- Jiraの閲覧権限とGitHubの修正権限が一致しない
- Grafana logs参照権限も慎重に扱う必要がある
- GitHub App / PAT / PR作成権限をどう扱うか
- AIがどこまで読めるか、どこまで書けるか

学び: 社内調整もアーキテクチャ

AIエージェントの設計は、コードだけでは完結しない。

- 誰の権限で動くのか
- どの情報を読めるのか
- どこまで書き込めるのか
- 監査ログは残るのか
- 人間の承認をどこで挟むのか
- 失敗時にどう追跡できるのか

権限、監査、通知、承認まで含めてアーキテクチャだった。

Slackの位置づけ

Slackは全件通知先ではなく、選定されたcaseの調査ワークスペースにする。

Jira Webhook

- > Qualixが対象判定
- > 対象外なら内部ログのみ
- > 対象ならSlackに初回投稿
- > 以後はthreadに調査結果を集約
- > 人間が承認
- > GitHub Issue / PR作成

全チケットを流すとnoisyになるため、Qualix側で選定してからSlackに出す。

Slack threadに集約したい情報

- Jira issue key / URL
- チケット要約
- 対象判定理由
- 推定 service / repository / environment
- QA host / branch / namespace の推定結果
- Grafana logs の要約
- 類似 Jira / GitHub Issue / PR
- 原因仮説
- 修正方針
- GitHub Issue / PR URL
- 人間の承認コメント
- Qualix の処理ステータス

Slack thread自体を調査ノートにする。

チケット更新時の問題

Jiraは `issue_created` だけでなく `issue_updated` も送る。

更新イベントを扱うなら、最初から冪等性を考えたい。

- Webhookはretryされる可能性がある
- 同じチケット更新で複数回処理が走ると困る
- 既存のSlack thread / GitHub Issueがあるなら新規作成したくない
- Qualix自身のJiraコメントを契機に再実行したくない

冪等性の方針

1. webhook delivery 単位で重複排除する
2. Jira issue 単位で case を一意に管理する
3. issue_updated は差分を見て、必要な時だけ再調査する
4. 既存 Slack thread / GitHub Issue / PR があれば新規作成せず追記する

保存したいもの:

- qualix_cases
- qualix_webhook_events
- qualix_case_events

case管理のイメージ

```
qualix_cases
```

- jira_issue_key
- status
- latest_jira_updated_at
- slack_channel_id
- slack_thread_ts
- github_issue_url
- github_pr_url
- automation_level

```
qualix_webhook_events
```

- webhook_identifier
- event_type
- jira_issue_key
- payload_hash
- status

まずは「同じdeliveryを二重処理しない」「同じissueでcaseを再利用する」だけでもかなり安全になる。

段階的な自動化

いきなり完全自動PRではなく、信頼できる範囲を広げる。

```
notify_only:  
  Slack通知だけ
```

```
investigate:  
  調査結果をSlack threadに投稿
```

```
create_issue_with_approval:  
  Slack承認後にGitHub Issue作成
```

```
create_issue:  
  GitHub Issueを自動作成
```

```
create_pr_with_approval:  
  Slack承認後にPR作成
```

```
create_pr:  
  低リスク条件のみPR自動作成
```

横展開・OSS化を見据えた課題

backendだけでなく、車載、決済、支払い請求基盤、IoT基盤などにも広げたい。

ただし、各チームで差分が大きい。

- チケットの書き方
- repo構成
- ログの見方
- QA環境の作り方
- Slack通知先
- 承認フロー

コアは薄く、現場知識はYAML設定に寄せたい。

今後の実装優先度

1. case管理とwebhook冪等性
2. Slack thread集約
3. Slack上の承認フロー
4. issue_updated対応
5. 低リスクPR自動作成

特に最初は、以下を優先したい。

- `qualix_cases`
- `qualix_webhook_events`
- `X-Atlassian-Webhook-Identifier` の保存
- `issue_key`単位のcase upsert

まとめ

コード生成AIにより、実装速度はかなり上がった。

しかし、業務で価値を出すには、コードを書く前の仕事をAIに渡す必要がある。

Qualixは、Jiraの品管チケットを起点に、調査・原因推定・GitHub Issue / PR作成へつなげる試み。

難しかったのは、モデルよりも、文脈収集・権限・通知・承認・運用導線だった。

最後に

AIにコードを書かせること自体は、どんどん簡単になっている。

ただし、実際の業務で価値を出すには、チケット検知、文脈収集、repo推定、ログ確認、権限、通知、承認まで含めて設計する必要がある。

Qualixは、Jiraの品管チケットを起点に、その一連の流れをAIに渡していくための試みです。